

CS 440 Assignment 1 Report

Team Details

- Reuben Geronimo (`rg1090`)
- Orland Geronimo (`ogg9`)

Part 1a - Why the first move is east in Figure 8

The agent uses the freespace assumption: any cell not known to be blocked is treated as traversable. Therefore, at the first planning step the agent runs A* on a presumed-unblocked grid and follows one shortest presumed-unblocked path to the target.

In Figure 8, moving east is selected because it is on a shortest presumed-unblocked path under the initial information and the implementation uses deterministic tie handling (same `f` values are broken consistently). If multiple shortest first moves exist initially (for example, east and north), the tie-breaking policy and neighbor processing order select one deterministically; in our implementation that results in east.

Part 1b - Why the agent always terminates in finite time; move bound

Let the finite grid contain `U` unblocked cells in the true world.

1. **Finite-time termination:**

At each replanning episode, the agent either:

- reaches the target, or
- discovers at least one new blocked cell that invalidates the current presumed path, or
- concludes no presumed-unblocked path exists and terminates with failure.

Since the grid is finite, the number of distinct blockage discoveries is finite, and each search/planning episode is finite. Therefore the overall process terminates in finite time.

2. **Upper bound on number of moves:**

In any episode, the agent traverses a simple path over presumed-unblocked cells; this path length is at most `U-1` (cannot be longer than visiting unblocked cells without repetition in a shortest-path computation). The number of replans is also bounded by `U` in finite grids (each failed attempt advances knowledge and cannot continue indefinitely without new information). Hence a coarse upper bound on total executed moves is:

$$\begin{aligned} & \backslash[\\ & \backslash \text{moves} \leq U \cdot (U-1) < U^2. \\ & \backslash] \end{aligned}$$

So the number of moves until success/failure is bounded above by the square of the number of unblocked cells.

Part 2 - Effects of tie-breaking in Repeated Forward A*

We implemented both Repeated Forward A* variants:

- `max\_g`: break `f` ties in favor of larger `g`.
- `min\_g`: break `f` ties in favor of smaller `g`.

All experiments were run on the same 50 mazes (`101x101`, seed `42`).

Observations

- Found rate is identical: `26/50` for both.
- Expansion count is dramatically lower for `max\_g`:
 - Mean expanded: `8,920` (`max\_g`) vs `201,837` (`min\_g`)

- Median expanded: `8,790` (`max\_g`) vs `242,449` (`min\_g`)
- Per-maze comparison: `max\_g` better on `46/50`, ties `4/50`, `min\_g` better `0/50`
- Runtime mirrors expansions:
 - Mean runtime: `76.7 ms` (`max\_g`) vs `1,190.4 ms` (`min\_g`)
 - Median runtime: `80.9 ms` (`max\_g`) vs `1,445.2 ms` (`min\_g`)

Explanation

With equal `f = g+h`, preferring larger `g` tends to prefer states closer to the goal along the same `f` contour, which focuses search and reduces wavefront growth. Preferring smaller `g` tends to keep expansion near the start side of the contour, increasing frontier breadth and therefore expansions/runtime.

Part 3 - Repeated Forward vs Repeated Backward A*

We compared:

- Repeated Forward A* (`max\_g`)
- Repeated Backward A* (`max\_g`)

Observations

- Found rate is identical: `26/50` for both.
- Forward was better on expansions for every maze:
 - Mean expanded: `8,920` (forward) vs `102,180` (backward)
 - Median expanded: `8,790` (forward) vs `101,170` (backward)
 - Wins on expanded: forward better `50/50`
- Runtime also favored forward on every maze:
 - Mean runtime: `77.1 ms` (forward) vs `1,325.6 ms` (backward)
 - Median runtime: `82.2 ms` (forward) vs `1,283.9 ms` (backward)

Explanation

In this partially observable setting, knowledge grows around the current agent location. Planning forward from the current state to goal aligned better with this evolving local knowledge than planning backward from the fixed goal toward a moving current state, resulting in fewer expansions and lower runtime in our 50-maze sample.

Part 4a - Manhattan distance is consistent in 4-neighbor grids

Let the heuristic be Manhattan distance to the goal:

$$h(s) = |x_s - x_g| + |y_s - y_g|.$$

For any neighboring states `s` and `s'` (one move in N/S/E/W), we have:

$$|x_s - x_{s'}| + |y_s - y_{s'}| = 1.$$

By triangle inequality on each coordinate:

$$h(s) \leq 1 + h(s').$$

Since each move cost is `c(s,s')=1` for traversable moves, this is exactly:

$$h(s) \leq c(s,s') + h(s').$$

Therefore Manhattan distance is consistent.

Part 4b - Why Adaptive A* keeps heuristics admissible and consistent under co

st increases

Adaptive A* updates expanded states after each search using:

$$h_{\text{new}}(s) = g(s_{\text{goal}}) - g(s).$$

For expanded states in that search, $g(s)$ is the shortest distance from the current search start to s , so $h_{\text{new}}(s)$ equals a valid lower bound on distance from s to the goal in that search instance. Thus updated values are admissible at update time.

If action costs later increase (never decrease), true shortest-path distances cannot decrease. Therefore any previously admissible lower bound remains admissible after increases.

For consistency: in the search where values are updated, h_{new} corresponds to shortest-path distances and satisfies the triangle inequality over one-step transitions. Non-updated states retain old consistent values. Because updates only raise informative lower bounds derived from shortest-path structure and costs only increase, the consistency inequalities are preserved across iterations (standard Adaptive A* result).

Hence Adaptive A* keeps initially consistent heuristics consistent and admissible even when costs increase.

Part 5 - Repeated Forward A* vs Adaptive A*

We compared:

- Repeated Forward A* (max_g)
- Adaptive A* (max_g)

Observations

- Found rate is identical: $26/50$ for both.
- Adaptive A* improved expansions on most mazes:
 - Mean expanded: $8,920$ (forward) vs $8,549$ (adaptive)
 - Median expanded: $8,790$ (forward) vs $8,717$ (adaptive)
 - Per-maze: adaptive better 34 , ties 14 , worse 2
- Runtime improvement was small and not consistently directional:
 - Mean runtime: 76.0 ms (forward) vs 74.6 ms (adaptive)
 - Median runtime: 81.4 ms (forward) vs 78.6 ms (adaptive)

Explanation

Adaptive A* reuses search experience by increasing heuristic values for expanded states using:

$$h_{\text{new}}(s) = g(s_{\text{goal}}) - g(s).$$

This generally focuses future replans and reduces expansions. Runtime gains are smaller because Python/runtime overhead and system noise can dominate at small absolute times.

Part 6 - Statistical significance plan and application

For paired maze-by-maze comparisons, we use a **two-sided sign test** on per-maze differences (primary metric: expanded nodes).

For each maze i , define:

$$d_i = \text{metric}_{\{B,i\}} - \text{metric}_{\{A,i\}}.$$

\]

- ``B`` wins if ``d_i < 0``
- ``B`` loses if ``d_i > 0``
- ties (``d_i=0``) are excluded

Under null hypothesis $(H_0: P(\text{B wins})=0.5)$, exact p-values come from `Binomial(n, 0.5)`, with `n = wins + losses`.

Results on regenerated 50-maze runs (expanded)

- Part 2 (``min_g`` vs ``max_g``): wins=0, losses=46, ties=4
 $(p \approx 2.84 \times 10^{-14})$
- Part 3 (``bwd`` vs ``fwd``): wins=0, losses=50, ties=0
 $(p \approx 1.78 \times 10^{-15})$
- Part 5 (``adaptive`` vs ``fwd``): wins=34, losses=2, ties=14
 $(p \approx 1.94 \times 10^{-8})$

At significance level $(\alpha=0.05)$, all three expanded-node differences are statistically significant.

Runtime note

For runtime in Part 5, sign test gives wins=25, losses=25, ties=0, so $(p=1.0)$: no statistically significant paired runtime advantage despite small mean/median differences.